Department of Computer Engineering

Technical Interview Question & Model Answer Bank

Academic Year	**2026-2027**	**Class**	**4th year**
Subject/Expert Area	**Data Structure & Algorithm**		
Expert Name	**Mr. A. K Sahu**		

Sr. No.	*Question*	*Model Answer*	*Practical Example*
1	*What is a data structure?*	*A data structure is basically a way of organizing and storing data so that we can use it efficiently. Different problems need different ways of arranging data.*	*For example, an array is great for storing student marks because everything sits in continuous memory and we can access any mark instantly.*
2	*What is an algorithm?*	*An algorithm is just a step-by-step procedure that tells us how to solve a particular problem. It's like a recipe for the computer.*	*Binary Search is a classic algorithm – it helps us find an element in a sorted array very quickly by repeatedly cutting the search space in half.*
3	*What is the difference between an array and a linked list?*	*The main difference is how they store data. An array keeps elements in continuous memory locations, so we get fast random access. A linked list stores elements in separate nodes that are connected by pointers, so insertion and deletion become easier, but we lose direct access by index.*	*If the number of students is fixed, an array works well. If students keep getting added or removed frequently, a linked list is more flexible.*
4	*What is the main advantage of an array?*	*The biggest advantage is that we can access any element in constant time just by using its index. We don't have to traverse anything.*	*So marks[3] immediately gives us the fourth student's marks – no searching required.*
5	*What is the disadvantage of an array?*	*The size is fixed when we create it. Also, if we want to insert or delete something in the middle, we usually have to shift a lot of elements, which is costly.*	*If I want to insert a new student at position 2, every student after that position has to be shifted one place ahead.*
6	*What is a linked list?*	*A linked list is a collection of nodes where each node holds some data and a pointer (or reference) to the next node. The nodes don't have to be stored next to each other in memory.*	*A music playlist is a good real-world example – each song points to the next one, and we can easily add or remove songs without rearranging everything.*
7	*What is a stack?*	*A stack is a linear data structure that follows the LIFO principle – Last In, First Out. The last element we push is the first one we pop.*	*The Undo feature in a text editor works like a stack. The most recent change is the first one that gets undone.*
8	*What is a queue?*	*A queue is a linear data structure that follows FIFO – First In, First Out. Elements are added at the rear and removed from the front, just like a real-life queue.*	*People waiting in line for movie tickets are a perfect example – the person who comes first gets the ticket first.*
9	*What is the difference between stack and queue?*	*The core difference is the order. A stack is LIFO (last element in is the first one out), while a queue is FIFO (first element in is the first one out).*	*Browser back button behaves like a stack. A printer job queue behaves like a queue – jobs are printed in the order they were submitted.*

10	*What are the basic operations of a stack?*	*The main operations are Push (add an element), Pop (remove the top element), Peek or Top (look at the top element without removing it), and checking whether the stack is empty or full.*	*Push puts a new item on top; Pop takes the top item off.*
11	*What are the basic operations of a queue?*	*The basic operations are Enqueue (add at the rear), Dequeue (remove from the front), and looking at the Front and Rear elements.*	*When a new customer joins the line we enqueue them. When the first customer is served we dequeue them.*
12	*What is stack overflow?*	*Stack overflow happens when we try to push an element onto a stack that is already full – there's simply no space left.*	*If we have an array-based stack of size 5 and we try to push a sixth element, we get a stack overflow.*
13	*What is stack underflow?*	*Stack underflow is the opposite – it happens when we try to pop an element from a stack that is already empty.*	*Calling Pop on an empty stack causes underflow because there's nothing to remove.*
14	*What is a circular queue?*	*A circular queue is a queue where the last position connects back to the first position, forming a circle. This way we can reuse empty spaces that appear at the front after dequeuing.*	*CPU scheduling often uses a circular queue so that processes keep getting turns in a round-robin fashion.*
15	*What is a tree?*	*A tree is a non-linear hierarchical data structure made up of nodes connected by edges. There's one root node, and every other node has exactly one parent.*	*A company's organizational chart is a classic tree – CEO at the top, then managers, then employees under them.*
16	*What is a binary tree?*	*A binary tree is a tree in which every node can have at most two children – usually called the left child and the right child.*	*Expression trees that represent mathematical expressions are binary trees, because operators typically take two operands.*
17	*What is a Binary Search Tree?*	*A Binary Search Tree is a binary tree with a special ordering property: for every node, all values in the left subtree are smaller, and all values in the right subtree are larger.*	*If we store student IDs in a BST, searching for a particular ID becomes very efficient – we keep going left or right based on comparison.*
18	*Which traversal of a BST gives sorted order?*	*Inorder traversal of a Binary Search Tree gives the elements in ascending (sorted) order. That's one of the nicest properties of a BST.*	*If the BST contains 20, 10 and 30, an inorder walk will visit them as 10, 20, 30.*
19	*What are tree traversal techniques?*	*The common ones are Preorder (root → left → right), Inorder (left → root → right), Postorder (left → right → root), and Level Order (level by level, usually using a queue).*	*When we want to process every file inside a folder structure, we traverse the tree of directories and files.*
20	*What is a heap?*	*A heap is a complete binary tree that satisfies the heap property. In a max-heap every parent is greater than or equal to its children; in a min-heap every parent is smaller than or equal to its children.*	*Priority queues are almost always implemented using heaps because we can get the highest (or lowest) priority element in logarithmic time.*
21	*What is a graph?*	*A graph is a non-linear data structure made of vertices (nodes) and edges that connect them. It's used whenever we need to represent relationships between objects.*	*A social network is a graph – each person is a vertex and each friendship is an edge.*
22	*What is the difference between BFS and DFS?*	*BFS explores the graph level by level and normally uses a queue. DFS goes as deep as possible along one path before backtracking, and it uses a stack (or recursion).*	*BFS is great for finding the shortest path in an unweighted graph. DFS is useful when we just need to explore all connected components.*

23	What is searching?	Searching is simply the process of looking for a particular element inside a collection of data.	Looking up a student's roll number in a list of students is a searching operation.
24	What is linear search?	Linear search checks every element one by one from the beginning until it either finds the target or reaches the end of the list. It works on both sorted and unsorted data.	If the list of student names is not sorted, the only practical way is to scan from the start until we find the name we want.
25	What is binary search?	Binary search works only on sorted data. It repeatedly divides the search space in half and discards the half that cannot contain the target.	Searching for a roll number among 10,000 sorted student records becomes extremely fast with binary search.
26	What is the time complexity of binary search?	Binary search runs in O(log n) time. Every comparison cuts the remaining search space roughly in half.	Even with a million sorted records, we only need about 20 comparisons in the worst case – much better than checking every record.
27	What is sorting?	Sorting means arranging data in a particular order – usually ascending or descending – so that it becomes easier to search or process later.	We sort student marks from highest to lowest when we prepare a merit list.
28	What is the difference between Bubble Sort and Selection Sort?	Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. Selection Sort finds the minimum (or maximum) element and places it at the correct position in each pass.	Both are simple and work fine for a small list of student marks, but neither is efficient for large data.
29	What is Merge Sort?	Merge Sort is a classic Divide-and-Conquer algorithm. It keeps dividing the array into halves until each piece has one element, then merges those sorted pieces back together.	It's a reliable choice when we need to sort a large collection of records and want guaranteed O(n log n) performance.
30	What is Quick Sort?	Quick Sort is also Divide-and-Conquer. It picks a pivot, partitions the array so that smaller elements are on one side and larger ones on the other, then recursively sorts the two sides.	In practice it's one of the fastest sorting algorithms for large arrays, as long as the pivot is chosen reasonably well.
31	What is time complexity?	Time complexity tells us how the running time of an algorithm grows as the input size grows. We usually express it using Big-O notation.	Linear Search is O(n) because in the worst case we may have to look at every single element.
32	What is space complexity?	Space complexity tells us how much extra memory an algorithm needs as the input size grows.	If an algorithm creates an extra array of size n, we say it has O(n) extra space.
33	What is Big-O notation?	Big-O notation describes the upper bound on the growth rate of an algorithm's time or space requirement. It helps us compare algorithms without worrying about exact constants.	An $O(n^2)$ algorithm will eventually become much slower than an O(n) algorithm as the input gets larger.
34	What is recursion?	Recursion is a technique where a function solves a problem by calling itself on a smaller version of the same problem, until it reaches a simple base case.	Calculating the factorial of a number is a classic example – $n! = n \times (n-1)!$ with the base case $0! = 1$.
35	What is the base condition in recursion?	The base condition is the stopping point – the simple case that doesn't need another recursive call. Without it the recursion would go on forever.	In factorial we usually write something like: if n equals 0, return 1.
36	What is Dynamic Programming?	Dynamic Programming is a technique we use when a problem has overlapping subproblems. Instead of solving the same subproblem again and again, we store the results and reuse them.	The Fibonacci sequence is the classic example – a plain recursive solution recalculates the same values many times, while DP calculates each value only once.

37	*What is the Greedy algorithm?*	*A Greedy algorithm makes the best choice that looks optimal at the current step, hoping that this will lead to a globally optimal solution. It never goes back to reconsider earlier decisions.*	*In the Activity Selection problem we always pick the activity that finishes earliest – that simple greedy choice actually gives the optimal schedule.*
38	*What is Divide and Conquer?*	*Divide and Conquer breaks a problem into smaller independent subproblems, solves each of them, and then combines the results to get the final answer.*	*Merge Sort is a perfect example – we divide the array, sort the halves, and then merge the sorted halves.*
39	*What is Backtracking?*	*Backtracking is a trial-and-error approach. We try a possible choice, and if it doesn't lead to a solution we undo that choice (backtrack) and try something else.*	*The N-Queens problem is usually solved with backtracking – we place a queen, and if we get stuck we remove it and try a different position.*
40	*What is hashing?*	*Hashing is a technique that maps a key to a specific location in a table using a hash function, so we can look up the data in nearly constant time.*	*We can convert a student roll number into an index with a hash function and jump straight to that student's record.*
41	*What is a doubly linked list?*	*A doubly linked list is like a normal linked list, but each node also has a pointer to the previous node. So we can move both forward and backward easily.*	*Browser history with Back and Forward buttons is a natural fit for a doubly linked list.*
42	*What is a circular linked list?*	*In a circular linked list the last node points back to the first node, so the list forms a closed loop. There is no real 'end'.*	*In a multiplayer game we can use a circular list so that after the last player's turn, it automatically goes back to the first player.*
43	*What is the difference between singly and doubly linked list?*	*A singly linked list only has a next pointer, so we can only move forward. A doubly linked list has both next and previous pointers, so we can move in either direction. Deletion is also simpler in a doubly linked list because we already know the previous node.*	*If we need to frequently delete a given node, a doubly linked list saves us the effort of finding the previous node first.*
44	*What is a priority queue?*	*A priority queue is a special kind of queue where every element has a priority. Higher-priority elements are served before lower-priority ones, regardless of the order they arrived.*	*In a hospital emergency room, critical patients are treated first even if they arrived later – that's a priority queue in action.*
45	*What is the difference between a queue and a priority queue?*	*A normal queue strictly follows FIFO – whoever comes first is served first. A priority queue serves elements according to their priority, not arrival time.*	*Print jobs usually go into a normal queue. CPU process scheduling often uses a priority queue so important processes run sooner.*
46	*What is an AVL tree?*	*An AVL tree is a self-balancing Binary Search Tree. It makes sure that the height difference between the left and right subtrees of any node is never more than one. Whenever the balance is disturbed by an insertion or deletion, it performs rotations to restore balance.*	*We use AVL trees when we need guaranteed fast search, insertion and deletion even after many updates.*
47	*What is the height of a tree?*	*The height of a tree is the number of edges on the longest path from the root down to a leaf. A single-node tree has height 0.*	*If the tallest branch from the root has four edges, we say the tree's height is 4.*
48	*What is the difference between a tree and a graph?*	*A tree is a special kind of graph that has no cycles and exactly one path between any pair of nodes. A general graph can have cycles and multiple paths between nodes.*	*A family tree is a tree. A city's road network is a graph – you can often reach the same place by different routes.*

49	What is an adjacency matrix?	An adjacency matrix is a 2-D array where matrix[i][j] is 1 (or the weight) if there is an edge from vertex i to vertex j, and 0 otherwise. Checking whether an edge exists is very fast – just one array lookup.	For a small dense graph of cities, an adjacency matrix lets us instantly check whether there is a direct road between any two cities.
50	What is an adjacency list?	An adjacency list stores, for each vertex, a list of all its neighbors. It is much more memory-efficient than a matrix when the graph is sparse (few edges).	Social networks with millions of users almost always use adjacency lists because most people are friends with only a small fraction of all users.
51	What is collision in hashing?	A collision happens when two different keys produce the same hash value and therefore want to go into the same slot in the hash table.	If roll numbers 25 and 45 both hash to index 5, we have a collision at that index.
52	What are the common methods to resolve collisions in hashing?	The two most common approaches are Chaining (each slot holds a linked list of all keys that hashed there) and Open Addressing (we probe for another empty slot – linear probing, quadratic probing, or double hashing).	With chaining, all student records that hash to the same index simply form a small linked list at that index.
53	What is the time complexity of Bubble Sort, Selection Sort, and Insertion Sort?	All three have $O(n^2)$ average and worst-case time complexity. Bubble Sort and Insertion Sort can become $O(n)$ in the best case when the array is already sorted.	These algorithms are fine for a few hundred elements, but they become painfully slow for large data sets.
54	What is the time complexity of Merge Sort and Quick Sort?	Merge Sort is always $O(n \log n)$ – best, average and worst case. Quick Sort is $O(n \log n)$ on average, but can degrade to $O(n^2)$ if the pivot choices are consistently bad.	When we need a guaranteed $O(n \log n)$ sort, Merge Sort is the safer choice.
55	What is the difference between a stable and an unstable sorting algorithm?	A stable sort keeps the relative order of equal elements the same as they appeared in the original input. An unstable sort does not give that guarantee.	Merge Sort is stable; the common versions of Quick Sort are not. Stability matters when we sort records by one key and then by another.
56	What is the difference between BFS and Dijkstra's algorithm?	BFS finds the shortest path in terms of number of edges (unweighted or equal-weight graphs). Dijkstra's algorithm finds the shortest path when edges have different non-negative weights.	BFS can tell us the minimum number of flights needed. Dijkstra is needed when the ticket prices are different and we want the cheapest route.
57	What is memoization?	Memoization is the technique of storing the results of expensive function calls so that if the same inputs come again, we just return the cached result instead of recomputing it. It's the heart of top-down Dynamic Programming.	In a recursive Fibonacci function we keep an array; once we have calculated F(k) we store it and never calculate it again.
58	What is the difference between recursion and iteration?	Recursion solves a problem by calling itself on a smaller input. Iteration uses loops. Recursion is often more elegant but uses extra stack space; iteration usually uses constant extra space.	Factorial can be written either as a recursive function or as a simple for-loop that multiplies numbers from 1 to n.
59	What is a complete binary tree?	A complete binary tree is one in which every level is completely filled except possibly the last level, and the last level is filled from left to right with no gaps.	Heaps are always complete binary trees, which is why we can store them efficiently in a plain array.
60	What is the difference between	A binary tree only restricts the number of children (at most two). A Binary Search Tree	Any hierarchical structure can be a binary tree. When we also need fast ordered search,

	a binary tree and a Binary Search Tree?	*adds the ordering rule: left subtree values are smaller, right subtree values are larger.*	*we use a BST.*
61	*When would you use a linked list instead of an array?*	*I would choose a linked list when the size of the data changes a lot, or when I need to insert or delete elements frequently at arbitrary positions. Arrays are better when I need fast random access by index.*	*A live music playlist where songs can be added or removed anywhere is more natural with a linked list.*
62	*What is the advantage of using a circular queue over a linear queue?*	*In a linear array-based queue, once we dequeue from the front the empty slots are wasted. A circular queue reuses those slots by wrapping around, so we make full use of the available space.*	*In an array of size 5, after removing two elements a circular queue can still accept new elements in the freed positions at the beginning.*
63	*What is topological sorting?*	*Topological sorting is a linear ordering of the vertices of a Directed Acyclic Graph such that for every directed edge u → v, u appears before v in the ordering. It only works on DAGs.*	*Determining the correct order to take college courses that have prerequisites is a classic topological-sort problem.*
64	*What is the difference between a min-heap and a max-heap?*	*In a min-heap the parent is always smaller than or equal to its children, so the root is the minimum. In a max-heap the parent is always greater, so the root is the maximum.*	*Dijkstra's algorithm uses a min-heap to always pick the next closest unvisited node. A max-heap is useful when we always want the highest-priority item.*
65	*What is the load factor in hashing?*	*Load factor is simply the ratio of the number of elements currently stored to the total number of slots in the hash table (n/m). It tells us how full the table is; a high load factor means more collisions.*	*If a table of size 100 holds 70 elements, the load factor is 0.7. Most implementations resize when the load factor gets too high.*
66	*What are the two main approaches in Dynamic Programming?*	*There are two popular ways: Top-Down (also called Memoization) and Bottom-Up (also called Tabulation). Top-Down starts from the original problem and uses recursion while caching results. Bottom-Up starts from the smallest subproblems and iteratively builds up to the final answer.*	*For Fibonacci we can either write a recursive function that remembers previous results, or simply fill an array from 0 up to n.*
67	*What is the difference between Memoization and Tabulation?*	*Memoization is top-down – we start from the big problem and recurse, storing results as we go. Tabulation is bottom-up – we start from the base cases and fill a table iteratively until we reach the answer. Memoization uses the call stack; Tabulation usually needs less stack space.*	*Both can compute the nth Fibonacci number, but one does it with recursion + cache, the other with a simple loop that fills an array.*
68	*What are overlapping subproblems?*	*Overlapping subproblems means that while solving a larger problem we end up solving the exact same smaller subproblem multiple times. Dynamic Programming saves the answer the first time so we never recompute it.*	*In Fibonacci, F(5) needs F(4) and F(3); F(4) also needs F(3). Without memoization, F(3) gets calculated more than once.*
69	*What is optimal substructure?*	*Optimal substructure means that the best solution to the whole problem can be built from the best solutions of its subproblems. If a problem has this property, Dynamic Programming can work.*	*The shortest path from A to C that goes through B is simply the shortest path from A to B plus the shortest path from B to C.*
70	*When should you use Dynamic*	*We should switch to Dynamic Programming whenever the recursive solution keeps solving the*	*Fibonacci and 0/1 Knapsack benefit hugely from DP. Merge Sort doesn't need DP*

	Programming instead of plain recursion?	*same subproblems over and over again. If the subproblems don't overlap, plain recursion or Divide-and-Conquer is usually enough.*	*because its subproblems never overlap.*
71	*What is the 0/1 Knapsack problem?*	*We are given a set of items, each with a weight and a value, and a knapsack with a maximum weight capacity. Each item can be taken at most once (either 0 or 1). The goal is to choose a subset of items that maximizes total value without exceeding the capacity.*	*A thief with a bag that can hold only 10 kg has to decide which whole items to steal to maximize the profit.*
72	*What is the difference between 0/1 Knapsack and Fractional Knapsack?*	*In 0/1 Knapsack we either take the whole item or leave it – no fractions allowed. In Fractional Knapsack we are allowed to take a fraction of an item. 0/1 needs Dynamic Programming; Fractional can be solved optimally with a simple Greedy approach.*	*Taking whole gold bars is 0/1. Taking gold dust that can be measured out in any quantity is Fractional.*
73	*How do you find the Longest Common Subsequence (LCS) using DP?*	*We build a 2-D table where dp[i][j] stores the length of the LCS of the first i characters of the first string and the first j characters of the second string. If the characters match we take the diagonal value plus one; otherwise we take the maximum of the cell above or the cell to the left.*	*For the strings 'AGGTAB' and 'GXTXAYB' the LCS is 'GTAB' and its length is 4.*
74	*What is the difference between subsequence and substring?*	*A subsequence is formed by deleting some (or no) characters without changing the order of the remaining ones – the characters don't have to be consecutive. A substring is a contiguous sequence of characters.*	*'ACE' is a subsequence of 'ABCDE'. 'BCD' is a substring of 'ABCDE'.*
75	*How can the Fibonacci sequence be solved using Dynamic Programming?*	*The simplest bottom-up way is to create an array where dp[0] = 0, dp[1] = 1, and then for every i from 2 to n we set dp[i] = dp[i-1] + dp[i-2]. The answer is simply dp[n]. Each number is calculated only once.*	*To get F(6) we end up with the array [0, 1, 1, 2, 3, 5, 8], so the answer is 8.*
76	*What is the time and space complexity of the bottom-up DP solution for Fibonacci?*	*Time is O(n) because we fill each of the n cells once. Space is also O(n) if we keep the whole array, but we can easily reduce it to O(1) by remembering only the previous two values.*	*Instead of the full array we can just keep two variables – prev2 and prev1 – and update them as we go.*
77	*What is the Coin Change problem in Dynamic Programming?*	*Given a set of coin denominations and a target amount, we want either the minimum number of coins needed to make that amount, or the number of different ways to make it. It's a classic unbounded-knapsack style DP problem.*	*With coins [1, 5, 10, 25] and amount 30, the fewest coins we need is two – one 25 and one 5.*
78	*What is the difference between Dynamic Programming and Greedy*	*A Greedy algorithm always picks the choice that looks best right now and never looks back. Dynamic Programming considers all possibilities by solving every overlapping subproblem, and therefore guarantees the global optimum when the*	*Fractional Knapsack can be solved correctly by Greedy. 0/1 Knapsack generally needs the full power of DP.*

	algorithms?	*problem has optimal substructure.*	
79	*What is the difference between Dynamic Programming and Divide and Conquer?*	*Both break a problem into smaller pieces. The key difference is that Divide-and-Conquer subproblems are independent, while Dynamic Programming subproblems overlap. DP stores results to avoid recomputation; Divide-and-Conquer doesn't need to.*	*Merge Sort is pure Divide-and-Conquer. Fibonacci is Dynamic Programming because the same subproblems keep appearing.*